

NumPy Master Practice Guide

Comprehensive Problem Set & Solutions: Very Basics → Basics → Intermediate Level

- 75 Hands-On Questions
- 3 Progression Levels
- Complete Solution Key
- Quick Syntax Cheat Sheet

💡 NumPy Quick Reference Cheat Sheet

Category	Common Functions & Methods	Example Usage
Array Creation	<code>array</code> , <code>zeros</code> , <code>ones</code> , <code>full</code> , <code>arange</code> , <code>linspace</code> , <code>eye</code>	<code>np.arange(0, 10, 2)</code> <code>np.linspace(0, 1, 5)</code>
Inspection	<code>shape</code> , <code>ndim</code> , <code>size</code> , <code>dtype</code> , <code>astype</code> , <code>nbytes</code>	<code>arr.shape</code> <code>arr.astype(np.int32)</code>
Indexing & Slicing	<code>arr[r, c]</code> , <code>arr[start:stop:step]</code> , <code>arr[mask]</code>	<code>arr[1:3, :]</code> <code>arr[arr > 5]</code>
Reshaping & Joining	<code>reshape</code> , <code>flatten</code> , <code>ravel</code> , <code>vstack</code> , <code>hstack</code> , <code>concatenate</code>	<code>arr.reshape(3, 4)</code> <code>np.vstack((a, b))</code>
Math & Stats	<code>sum</code> , <code>mean</code> , <code>std</code> , <code>min</code> , <code>max</code> , <code>argmin</code> , <code>argmax</code> , <code>dot</code>	<code>arr.sum(axis=0)</code> <code>np.dot(A, B)</code>
Conditionals & Logic	<code>where</code> , <code>clip</code> , <code>unique</code> , <code>intersect1d</code> , <code>isnan</code>	<code>np.where(arr > 0, 1, -1)</code>

Part 1: Very Basics (Array Creation & Inspection) QUESTIONS 1 – 25

- Q1. Import the NumPy library using the standard alias `np` and print the installed version.
- Q2. Create a 1D NumPy array from the Python list `[10, 20, 30, 40, 50]` and print its type.
- Q3. Generate a 1D array containing integers from 10 to 49 (inclusive) using `np.arange()`.
- Q4. Create a 3x3 matrix filled entirely with floating-point zeros.
- Q5. Create a 2x5 matrix filled entirely with integer ones.
- Q6. Create a 4x4 matrix initialized with the constant value `7` using `np.full()`.
- Q7. Create a 4x4 Identity Matrix using `np.eye()`.
- Q8. Create an array of 10 evenly spaced values between 0 and 1 (inclusive) using `np.linspace()`.
- Q9. Given `arr = np.array([[1, 2, 3], [4, 5, 6]])`, print its `shape`, number of dimensions (`ndim`), total number of elements (`size`), and data type (`dtype`).
- Q10. Convert an array `arr = np.array([1.5, 2.7, 3.8, 4.2])` from float to 32-bit integer (`int32`) using `astype()`.

Q11. Create a 1D array of numbers from 1 to 12 and reshape it into a 3x4 2D matrix.

Q12. Calculate the total memory size (in bytes) consumed by a NumPy array with 100 `float64` elements using `nbytes` or `size * itemsize`.

Q13. Reverse a 1D array `arr = np.arange(10)` using slicing so that the first element becomes the last.

Q14. Create a 3D array with shape `(2, 3, 4)` populated with zeros.

Q15. Create a 3x3 boolean array with all elements initialized to `True`.

Q16. Generate a 1D array starting from 100 up to (but excluding) 200 with a step size of 15.

Q17. Check if an array `arr = np.array([0, 0, 5, 0])` contains any non-zero elements using `np.any()`.

Q18. Transpose a 2x3 matrix `arr = np.array([[1, 2, 3], [4, 5, 6]])` using `.T` or `np.transpose()`.

Q19. Create a 1D array of 5 zeros, then change the 3rd element (index 2) to `99`.

Q20. Check whether all elements in `arr = np.array([1, 2, 3, 4])` are non-zero using `np.all()`.

Q21. Generate a 1D array of 5 random floats uniformly distributed between 0 and 1 using `np.random.rand()`.

Q22. Create a 3x3 array of random integers between 10 and 50 (inclusive) using `np.random.randint()`.

Q23. Find the minimum and maximum values in `arr = np.array([45, 12, 89, 3, 67])`.

Q24. Compute the sum of all elements in `arr = np.array([10, 20, 30, 40, 50])` using `np.sum()`.

Q25. Demonstrate array copying: Make an explicit deep copy of `a = np.array([1, 2, 3])` into `b` using `.copy()`, modify `b[0] = 999`, and verify `a` remains unchanged.

Part 2: Basic Operations, Slicing & Masking QUESTIONS 26 – 50

Q26. Given `arr = np.array([10, 20, 30, 40, 50, 60, 70])`, extract elements at index positions 1, 3, and 5.

Q27. Given a 3x3 matrix `m = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])`, extract the element at row 2, column 3 (value 60).

Q28. From a 4x4 matrix of numbers 1 to 16, slice out the 2x2 sub-matrix located at the top-right corner.

Q29. Extract all odd numbers from `arr = np.arange(1, 11)` using boolean masking.

Q30. Replace all odd numbers in `arr = np.arange(1, 10)` with `-1` without modifying the original array (create a copy).

Q31. In an array `arr = np.arange(10)`, set all elements greater than 5 to `100` in-place.

Q32. Flatten a 2D matrix into 1D using both `ravel()` and `flatten()`. What is the key difference regarding memory?

Q33. Perform element-wise addition, subtraction, multiplication, and division between `a = np.array([10, 20, 30])` and `b = np.array([2, 4, 5])`.

Q34. Compute the square root (`np.sqrt`), exponential (`np.exp`), and natural log (`np.log`) of `[1, 4, 9, 16]`.

Q35. Given a 3x3 matrix, calculate column-wise sums (`axis=0`) and row-wise sums (`axis=1`).

Q36. Compute the mean, median, standard deviation, and variance of `[12, 25, 34, 48, 51, 67, 89]`.

Q37. Find the index of the minimum and maximum element in `arr = np.array([15, 3, 42, 88, 7, 23])` using `argmin()` and `argmax()`.

Q38. Count how many elements in `arr = np.array([10, 25, 30, 45, 50, 65])` are strictly greater than 30.

Q39. Compute element-wise remainder (modulo) when dividing `[10, 15, 20, 25, 30]` by 4 using `%` or `np.remainder()`.

Q40. Clip all values in `arr = np.array([3, 12, 45, 80, 120])` so they lie strictly within the range `[10, 50]` using `np.clip()`.

Q41. Round float elements in `arr = np.array([1.234, 5.678, 9.101])` to 2 decimal places using `np.round()`.

Q42. Compare two arrays `a = np.array([1, 2, 3])` and `b = np.array([1, 2, 4])` element-wise, and test if they are completely equal using `np.array_equal()`.

Q43. Create a 3x3 diagonal matrix with diagonal elements `[5, 10, 15]` using `np.diag()`.

Q44. Given a 3x3 matrix, extract its main diagonal values as a 1D array.

Q45. Concatenate two 1D arrays `a = np.array([1, 2, 3])` and `b = np.array([4, 5, 6])` using `np.concatenate()`.

Q46. Vertically stack two 2D arrays `A` (2x3) and `B` (2x3) using `np.vstack()`.

Q47. Horizontally stack two 2D arrays `A` (2x3) and `B` (2x3) using `np.hstack()`.

Q48. Split a 1D array of 12 elements into 4 equal sub-arrays using `np.split()` or `np.array_split()`.

Q49. Repeat each element of `[1, 2, 3]` three times to get `[1, 1, 1, 2, 2, 2, 3, 3, 3]` using `np.repeat()`.

Q50. Tile an entire array `[1, 2]` three times to produce `[1, 2, 1, 2, 1, 2]` using `np.tile()`.

Part 3: Intermediate NumPy & Applied Operations QUESTIONS 51 – 75

Q51. Broadcasting: Add a 1D array `b = np.array([10, 20, 30])` (shape `(3,)`) to a 2D matrix `A` of shape `(4, 3)`. Explain how broadcasting alignment works.

Q52. Min-Max Normalization: Normalize a 4x4 matrix of random floats so all values are scaled between 0 and 1: `norm = (X - X.min()) / (X.max() - X.min())`.

Q53. Standardization (Z-score): Standardize a 2D array by subtracting column means and dividing by column standard deviations: `Z = (X - mean) / std`.

Q54. Given `arr = np.array([10, 20, 30, 40, 50])`, use `np.where()` to replace values `< 25` with `0` and values `>= 25` with `1`.

Q55. Fancy Indexing: From a 5x5 matrix, extract elements at row positions `[0, 2, 4]` and column positions `[1, 3, 0]` simultaneously.

Q56. Matrix Multiplication: Multiply a 2x3 matrix `A` by a 3x2 matrix `B` using the `@` operator and `np.matmul()`.

Q57. Find common elements between two 1D arrays `a = np.array([1, 2, 3, 4, 5])` and `b = np.array([3, 4, 5, 6, 7])` using `np.intersect1d()`.

Q58. Find elements present in array `a` but NOT in array `b` using `np.setdiff1d()`.

Q59.
Extract unique elements and their frequencies/counts from `arr = np.array([1, 2, 2, 3, 3, 3, 4, 4, 4, 4])` using `np.unique(..., return_counts=True)`.

Q60. Generate 10 random samples from a Standard Normal Distribution (mean=0, std=1) using `np.random.randn()` after setting a seed of `42`.

Q61. Randomly sample 5 unique elements without replacement from `np.arange(20)` using `np.random.choice()`.

Q62. Find the indices of non-zero elements in `arr = np.array([1, 0, 2, 0, 3, 0, 0, 4])` using `np.nonzero()` or `np.argwhere()`.

Q63. Given `arr = np.array([1.0, np.nan, 2.5, np.nan, 4.0])`, count the number of `NaN` values and replace all `NaN` entries with the mean of non-`NaN` values.

Q64. Sort a 2D matrix by its second column (index 1) using `argsort()`.

Q65. Compute cumulative sum (`np.cumsum`) and cumulative product (`np.cumprod`) along columns for a 3x3 matrix.

Q66. Compute a moving average with window size 3 over a 1D array `[1, 2, 3, 4, 5, 6, 7, 8, 9]` using `np.convolve()`.

Q67. Compute the determinant (`np.linalg.det`) and inverse (`np.linalg.inv`) of a 2x2 matrix `[[4, 7], [2, 6]]`.

Q68. Calculate Euclidean distance between two 3D vectors `p1 = np.array([1, 2, 3])` and `p2 = np.array([4, 5, 6])` using `np.linalg.norm()`.

Q69. Given `arr = np.array([10, 20, 30, 40, 50])`, find the value in `arr` closest to a target scalar `v = 27`.

Q70. Create a 6x6 checkerboard matrix pattern (alternating zeros and ones) using slicing strides `::2, ::2`.

Q71. Compute the 25th, 50th (median), and 75th percentiles of a random 1D array using `np.percentile()`.

Q72. Convert a 2D matrix into a binary mask array where elements strictly greater than the matrix median are `1`, and others are `0`.

Q73. Expand array dimensions: Convert a 2D array of shape `(3, 3)` into 3D shapes `(1, 3, 3)` and `(3, 3, 1)` using `np.expand_dims()` or `np.newaxis`.

Q74. Squeeze array dimensions: Convert a 3D array of shape `(1, 4, 1)` down to a 1D array of shape `(4,)` using `np.squeeze()`.

Q75. Vector products: Compute the inner product (`np.inner`), outer product (`np.outer`), and cross product (`np.cross`) of 3D vectors `u = np.array([1, 2, 3])` and `v = np.array([4, 5, 6])`.

Solutions: Q1 to Q10 (Basics & Creation)

```
import numpy as np

# Q1: Version
print("NumPy Version:", np.__version__)

# Q2: Creation & Type
a2 = np.array([10, 20, 30, 40, 50])
print(type(a2)) # <class 'numpy.ndarray'>

# Q3: Arange
a3 = np.arange(10, 50)

# Q4: Zeros float
a4 = np.zeros((3, 3), dtype=float)

# Q5: Ones int
a5 = np.ones((2, 5), dtype=int)

# Q6: Full
a6 = np.full((4, 4), 7)

# Q7: Eye
a7 = np.eye(4)

# Q8: Linspace
a8 = np.linspace(0, 1, 10)

# Q9: Inspection
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape, arr.ndim, arr.size, arr.dtype) # (2,3), 2, 6, int64

# Q10: Astype
a10 = np.array([1.5, 2.7, 3.8, 4.2]).astype(np.int32) # [1, 2, 3, 4]
```

Solutions: Q11 to Q25 (Attributes & Basics)

```
# Q11: Reshape
a11 = np.arange(1, 13).reshape(3, 4)

# Q12: Memory size
a12 = np.zeros(100, dtype=np.float64)
print("Bytes:", a12.nbytes) # 800 bytes

# Q13: Reverse 1D
a13 = np.arange(10)[::-1]

# Q14: 3D Array
a14 = np.zeros((2, 3, 4))

# Q15: Boolean Full
a15 = np.full((3, 3), True, dtype=bool)

# Q16: Arange Step
a16 = np.arange(100, 200, 15)

# Q17: Any
a17 = np.any(np.array([0, 0, 5, 0])) # True

# Q18: Transpose
a18 = np.array([[1, 2, 3], [4, 5, 6]]).T

# Q19: Modify element
a19 = np.zeros(5)
a19[2] = 99

# Q20: All
a20 = np.all(np.array([1, 2, 3, 4])) # True

# Q21: Random uniform
a21 = np.random.rand(5)

# Q22: Random ints
a22 = np.random.randint(10, 51, size=(3, 3))

# Q23: Min & Max
a23 = np.array([45, 12, 89, 3, 67])
print(a23.min(), a23.max()) # 3, 89

# Q24: Sum
a24 = np.sum(np.array([10, 20, 30, 40, 50])) # 150

# Q25: Explicit Deep Copy
orig = np.array([1, 2, 3])
cpy = orig.copy()
cpy[0] = 999 # orig remains [1, 2, 3]
```

Solutions: Q26 to Q40 (Slicing, Math & Stats)

```
# Q26: Specific index extraction
arr26 = np.array([10, 20, 30, 40, 50, 60, 70])
res26 = arr26[[1, 3, 5]] # [20, 40, 60]

# Q27: 2D indexing
m27 = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
val = m27[1, 2] # 60

# Q28: Corner submatrix slice
m28 = np.arange(1, 17).reshape(4, 4)
top_right = m28[:2, 2:]

# Q29: Extract odds with mask
arr29 = np.arange(1, 11)
odds = arr29[arr29 % 2 != 0]

# Q30: Copy replace odds
arr30 = np.arange(1, 10)
res30 = np.where(arr30 % 2 != 0, -1, arr30)

# Q31: In-place mask replace
arr31 = np.arange(10)
arr31[arr31 > 5] = 100

# Q32: Ravel vs Flatten
# ravel() returns a 1D view (modifying it changes original)
# flatten() returns a 1D deep copy.

# Q33: Element-wise ops
a, b = np.array([10, 20, 30]), np.array([2, 4, 5])
add, sub, mul, div = a + b, a - b, a * b, a / b

# Q34: Ufuncs
vals = np.array([1, 4, 9, 16])
s, e, l = np.sqrt(vals), np.exp(vals), np.log(vals)

# Q35: Axis sums
m35 = np.arange(1, 10).reshape(3, 3)
col_sum = m35.sum(axis=0) # [12, 15, 18]
row_sum = m35.sum(axis=1) # [6, 15, 24]

# Q36: Descriptive Stats
arr36 = np.array([12, 25, 34, 48, 51, 67, 89])
mean, med, std, var = np.mean(arr36), np.median(arr36), np.std(arr36), np.var(arr36)

# Q37: Argmin / Argmax
arr37 = np.array([15, 3, 42, 88, 7, 23])
min_idx, max_idx = np.argmin(arr37), np.argmax(arr37) # 1, 3

# Q38: Count condition
arr38 = np.array([10, 25, 30, 45, 50, 65])
cnt = np.sum(arr38 > 30) # 3

# Q39: Modulo
rem = np.array([10, 15, 20, 25, 30]) % 4 # [2, 3, 0, 1, 2]

# Q40: Clip
clipped = np.clip(np.array([3, 12, 45, 80, 120]), 10, 50) # [10, 12, 45, 50, 50]
```

Solutions: Q41 to Q50 (Rounding, Stacking & Splitting)

```
# Q41: Round
rounded = np.round(np.array([1.234, 5.678, 9.101]), 2) # [1.23, 5.68, 9.10]

# Q42: Array equality
a, b = np.array([1, 2, 3]), np.array([1, 2, 4])
elem_eq = np.equal(a, b) # [True, True, False]
all_eq = np.array_equal(a, b) # False

# Q43: Diagonal Matrix
d_mat = np.diag([5, 10, 15])

# Q44: Extract Diagonal
diag_vals = np.diag(d_mat) # [5, 10, 15]

# Q45: Concatenate
c = np.concatenate([np.array([1, 2, 3]), np.array([4, 5, 6])])

# Q46: Vertical Stack
A = np.ones((2, 3))
B = np.zeros((2, 3))
v_stacked = np.vstack((A, B)) # shape (4, 3)

# Q47: Horizontal Stack
h_stacked = np.hstack((A, B)) # shape (2, 6)

# Q48: Array Split
splits = np.array_split(np.arange(12), 4)

# Q49: Repeat
rep = np.repeat([1, 2, 3], 3) # [1, 1, 1, 2, 2, 2, 3, 3, 3]

# Q50: Tile
tiled = np.tile([1, 2], 3) # [1, 2, 1, 2, 1, 2]
```

Solutions: Q51 to Q62 (Intermediate Broadcasting & Sampling)

```
# Q51: Broadcasting
A = np.zeros((4, 3))
b = np.array([10, 20, 30])
res51 = A + b # b is broadcast across all 4 rows!

# Q52: Min-Max Normalization
X = np.random.rand(4, 4)
X_norm = (X - X.min()) / (X.max() - X.min())

# Q53: Z-Score Standardization
data = np.random.randn(5, 3)
Z = (data - data.mean(axis=0)) / data.std(axis=0)

# Q54: np.where condition
arr54 = np.array([10, 20, 30, 40, 50])
res54 = np.where(arr54 < 25, 0, 1) # [0, 0, 1, 1, 1]

# Q55: Fancy Indexing
m55 = np.arange(25).reshape(5, 5)
fancy_res = m55[[0, 2, 4], [1, 3, 0]]

# Q56: Matrix Multiplication
m1 = np.ones((2, 3))
m2 = np.ones((3, 2))
prod = m1 @ m2 # Shape (2, 2)

# Q57 & Q58: Set Operations
a1 = np.array([1, 2, 3, 4, 5])
a2 = np.array([3, 4, 5, 6, 7])
common = np.intersect1d(a1, a2) # [3, 4, 5]
diff = np.setdiff1d(a1, a2) # [1, 2]

# Q59: Unique with counts
vals, counts = np.unique([1, 2, 2, 3, 3, 3], return_counts=True)

# Q60: Random seed & randn
np.random.seed(42)
samples = np.random.randn(10)

# Q61: Random choice without replacement
chosen = np.random.choice(np.arange(20), size=5, replace=False)

# Q62: Nonzero indices
nz_idx = np.nonzero([1, 0, 2, 0, 3, 0, 0, 4])[0] # [0, 2, 4, 7]
```

Solutions: Q63 to Q75 (Advanced Logic & Linear Algebra)

```
# Q63: Missing Data Handling (NaN)
nan_arr = np.array([1.0, np.nan, 2.5, np.nan, 4.0])
nan_count = np.isnan(nan_arr).sum() # 2
mean_val = np.nanmean(nan_arr)
nan_arr[np.isnan(nan_arr)] = mean_val

# Q64: Sort 2D by column
m64 = np.array([[3, 9], [1, 4], [2, 7]])
sorted_m = m64[m64[:, 1].argsort()]

# Q65: Cumsum & Cumprod
m65 = np.array([[1, 2], [3, 4]])
c_sum = np.cumsum(m65, axis=0)
c_prod = np.cumprod(m65, axis=0)

# Q66: Moving Average (Convolution)
data66 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
moving_avg = np.convolve(data66, np.ones(3)/3, mode='valid')

# Q67: Determinant & Inverse
mat2x2 = np.array([[4, 7], [2, 6]])
det = np.linalg.det(mat2x2)
inv = np.linalg.inv(mat2x2)

# Q68: Euclidean Distance
p1, p2 = np.array([1, 2, 3]), np.array([4, 5, 6])
dist = np.linalg.norm(p1 - p2)

# Q69: Find nearest value to target
v_arr = np.array([10, 20, 30, 40, 50])
target = 27
nearest = v_arr[np.abs(v_arr - target).argmin()] # 30

# Q70: Checkerboard Matrix
cb = np.zeros((6, 6), dtype=int)
cb[1::2, ::2] = 1
cb[:, 1::2] = 1

# Q71: Percentiles
p25, p50, p75 = np.percentile(np.random.rand(100), [25, 50, 75])

# Q72: Threshold Mask
m72 = np.random.rand(4, 4)
bin_mask = (m72 > np.median(m72)).astype(int)

# Q73: Expand Dims
arr_2d = np.zeros((3, 3))
e1 = np.expand_dims(arr_2d, axis=0) # (1, 3, 3)
e2 = np.expand_dims(arr_2d, axis=-1) # (3, 3, 1)

# Q74: Squeeze Dims
arr_3d = np.zeros((1, 4, 1))
squeezed = np.squeeze(arr_3d) # (4,)

# Q75: Vector Products
u, v = np.array([1, 2, 3]), np.array([4, 5, 6])
inner_p = np.inner(u, v) # 32
outer_p = np.outer(u, v) # (3, 3) matrix
cross_p = np.cross(u, v) # [-3, 6, -3]
```